

Compte-rendu d'activité : Robot Framework avec Django REST

Projet de tests automatisés

15 septembre 2025

1 Introduction : Robot Framework

Robot Framework est un **framework open source d'automatisation de tests** basé sur une syntaxe en texte clair (*keyword driven testing*). Ses points forts :

- Lisible par des non-développeurs grâce à une syntaxe proche du langage naturel.
- Extensible : bibliothèques officielles (HTTP, base de données, etc.) ou personnalisées.
- Supporte aussi bien les tests d'API, de web UI que d'acceptation.

Ici, nous l'utilisons pour tester une **API Django REST Framework** sécurisée par **JWT (SimpleJWT)**.

2 Préparation du projet

1. Lancer le serveur Django REST :

```
python manage.py runserver
```

2. Installer Robot Framework et la librairie HTTP :

```
pip install robotframework  
pip install robotframework-requests
```

3 Structure des tests

Deux suites de tests distinctes sont écrites :

- **users.robot** : scénarios autour des utilisateurs et de leurs permissions.
- **todos.robot** : scénarios de type CRUD (Create, Read, Update, Delete) sur les tâches.

Chaque suite suit la même architecture :

Settings Charge les librairies (`RequestsLibrary`, `JSONLibrary`, `Collections`) et définit un **Suite Setup** qui crée une session HTTP réutilisable.

Variables Centralise les URLs d'API et les identifiants de test.

Keywords Définit les actions réutilisables : login, création d'entité, mise à jour, suppression, etc.

Test Cases Liste les scénarios de test exécutables.

4 Suite `users.robot`

Fonctionnalités testées

- **Authentication** : obtention d'un *access token* et d'un *refresh token*.
- **Création d'utilisateur public** : via l'endpoint `/register/`.
- **Administration** : opérations réservées aux administrateurs :
 - Lister les utilisateurs (`GET /users/`)
 - Attribuer ou retirer le rôle `is_staff`
 - Mettre à jour l'email d'un utilisateur
 - Désactiver puis supprimer un compte

Organisation des tests

Valid Login + Refresh Token : vérifie que l'API renvoie bien les jetons JWT.

Users Non-Admin Flow : contrôle que l'accès à `/users/` renvoie bien 403 si le compte n'est pas admin.

Users Flow : parcours complet de listing (nécessite un compte admin).

Users Admin Flow : création d'un user, passage en staff, mise à jour d'email, désactivation et suppression, puis vérification du 404 final.

5 Suite `todos.robot`

Fonctionnalités testées

- **CRUD complet** sur les tâches :
 - **Create** : création d'une todo
 - **Read** : récupération d'une todo spécifique
 - **List** : liste des todos de l'utilisateur
 - **Update** : modification du titre et de l'état
 - **Delete** : suppression d'une todo
- **Validation métier** : l'API interdit `inprogress=True` et `completed=True` simultanément.
- **Filtres de recherche** : vérification de l'endpoint avec paramètre `?status=`.

Organisation des tests

Auth Smoke For Todos : simple test de connexion pour vérifier la récupération du JWT.

CRUD Todo End-To-End : enchaîne création, lecture, listing, mise à jour et suppression, puis contrôle le 404 post-suppression.

Todo Validation Error (Both True) : vérifie que l'API renvoie 400 si deux drapeaux incompatibles sont envoyés.

Todos Filtering By Status : crée plusieurs todos et vérifie les filtres open et completed.

6 Exécution des tests

Pour exécuter une suite :

```
robot testsRobot/users.robot
robot testsRobot/todos.robot
```

ou pour tout exécuter :

```
robot testsRobot
```

7 Sortie des tests

Après l'exécution, Robot Framework produit trois fichiers à la racine du projet :

- **output.xml** : résultat machine lisible.
- **log.html** : journal détaillé des étapes, captures et valeurs.
- **report.html** : rapport synthétique avec le nombre de tests passés/échoués.

8 Conclusion

Robot Framework permet de **tester de bout en bout** une API Django REST :

- les tests sont lisibles par tous (même non-développeurs),
- l'exécution est automatisable (CI/CD),
- les rapports HTML sont prêts à l'emploi.

Ce tutoriel montre comment automatiser les *use-cases* essentiels :

- gestion d'utilisateurs (création, rôles, suppression),
- gestion de tâches (CRUD complet, validation et filtres).

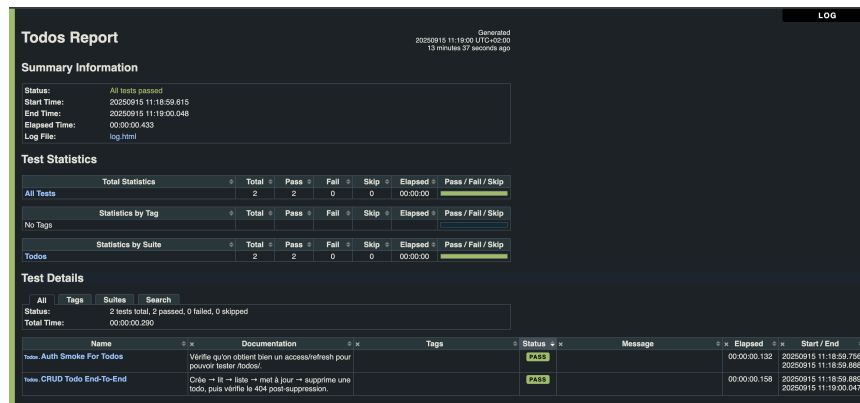


FIGURE 1 – Exemple d’écran du rapport HTML (log.html) montrant un test *CRUD Todo End-To-End* réussi.